

Unitree Week 33

Presentation by Mario Ortiz

This Week

- Completed Sim2Real Offline Tests to prepare for physical deployment.
- Researched steps needed for setup of the Unitree R1 as well as the R1 deployment steps for testing and verifying the connection between software, hardware, and firmware.

Gallantry Main Choices

Item	Cost	Pros
<u>Unitree Protective Gallantry for R1 Humanoid Robot</u>	\$1500	Unitree R1- Specific
<u>Pittsburgh 1-ton foldable shop crane</u>	\$279.99	Low Cost Option, not Unitree R1-Specific

Sim2Real Preparation

Sim2Real Physical R1 Validation

- Determining authoritative physical joint limits (rather than MuJoCo's limits)
- Reviewing Unitree's developer documentation, low-level examples, SDK definitions, and any robot-specific manuals/configuration files to ensure our physical joint limits align.

Information from Physical Robot

- R1 model: R1 EDU
 - Base is 26-DoF configuration
- Installed accessories / hands (extra 14 DOF)
- Robot firmware version
- Motion-control firmware/software version
- Remote-controller version
- SDK compatibility

Mechanical Safety Protocol

- R1 attached to approved support/gantry
 - Clear work area
 - Remote available
 - Physical recovery procedure understood
 - Emergency-stop behavior understood/tested
 - Operator positioned safely
 - Ethernet cable arranged so it cannot snag robot

Unitree R1 Standard Operation

- Before RL model deployment, test the R1's normal startup procedure
 - Power up
 - Normal Startup
 - Remote Communication
 - Zero Torque (or supported mode)
 - Normal shutdown

Setting up Ethernet and DDS

tools/read_only_lowstate/

└─ r1_read_only_lowstate

- This subscriber only program could be implemented onto the Unitree to create “rt/lowstate subscriber”. It will address these items regarding the robot:
 - real LowState data
 - Ethernet/DDS behavior
 - motor_state size
 - mode_machine
 - IMU quaternion
 - IMU gyro
 - real motor IDs — partially
- We must also verify if packets arrive continuously and are physically reasonable.

Setting up Ethernet and DDS

- Once that works we can extend the program long enough to characterize:
 - packet frequency
 - inter-arrival time
 - maximum observed gap
 - average period
 - jitter
 - dropped/stalled stream behavior
- The designed controller current allows approximately 1000ms. It is assumed the R1 will be faster than that.

Exact Motor IDs

- Ideally, while the R1 is securely supported and in a manufacturer-approved zero-torque/passive condition, we can observe which `motor_state[i].q` changes when a specific joint is moved.
- Example protocol:
 1. physically move the left knee slightly
 2. Check which LowState q changes
 3. Confirm the expected DDS ID (in this case, DDS ID = 3)

This shows the connection between real the physical joint, LowState motor ID , and our policy mapping.

Verifying the Physical IMU

What is still unknown until we use the physical robot is the firmware setup. With the IMU, we must obtain information about:

- Stationary upright
 - A parameter such as gyro $\approx [0, 0, 0]$
 - And a parameter indicating an upright quaternion consistent with the expected orientation.
- Roll motion
 - Gently rotate/support the robot around roll and verify which gyro component changes and its sign.
 - The same will be done for the Pitch and the Yaw motion.
-

Mode_machine Behavior

- During the standard manufacturer-controlled states, we should record the reported mode_machine parameters
 - Powered / zero torque
 - Supported
 - Stock standing mode
 - Debug mode
 - Any other information
- Then we can verify if the mode expected by our controller is appropriate.

Control Ownership

*Note that we do this before publishing LowCmd

- This is to ensure what owns low-level motor control (must ensure our controller doesn't take control from another service, which could break the system).
 1. Query ownership information
 2. Understand existing mode
 3. Explicit intentional transition
 4. Then we can safely enable our controller
- Important Parameters involve:
 - MotionSwitcher CheckMode
 - MotionSwitcher SelectMode
 - MotionSwitcher ReleaseMode
 - existing Unitree motion service
 - competing rt/lowcmd writers

Real Emergency/Recovery Path

- Before the first custom motor command, prove that we know how to stop it.
- This includes the manufacturer's physical/remote mechanism, not merely Ctrl+C.
- We should be able to answer:

Custom Command Test 1: damping/passive only

The first active test should essentially be the safest command our lifecycle controller supports (passive mode):

- $K_p = 0$; $K_d > 0$; $\tau = 0$; $dq_target = 0$

What we want to verify:

- LowCmd arrives
- No violent motion
- No incorrect joint activation
- Expected damping behavior
- Controller exits safely
- Ctrl+C shutdown works
- Physical emergency mechanism still works

Custom Command Test 1: damping/passive only

After damping: Commanded Joint Signs:

- We must confirm that **positive q command** moves the physical joint in the same direction expected by the policy.

Steps to do this are:

1. one joint
2. small position offset
3. low/reasonable gain
4. observe physical direction
5. observe LowState q
6. return to neutral

Watchman/Deadman Behavior

Once publishing is working, we test

- controller publishing
- stop publication, and view what does the R1 do.

We find out information such as if:

- firmware times out
- motors retain last command
- motor damping activates
- robot drops torque
- some other behavior occurs

Custom Command 2: FixStand

Now we can run command from Passive to FixStand state. We can then verify:

- all joints move in expected directions
- pose resembles simulated FixStand
- no joint approaches limits
- no oscillation
- no abnormal command
- no controller fault

Actual Joint Commands

We should compare actual joint commands against physical limits with the LowCmd again.

- We then decide whether controller requires an explicit clamp or not.

RL Policy Test 1: supported ground contact

Gantry setup for ground contact

1. Gantry supports the R1
2. Gradually lower
3. Feet contact floor
4. Gantry remains attached
5. Robot bears some/all weight
6. Gantry catches failure

RL Policy Test 2: Locomotion

We now initiate a small locomotion command

- Set the command for only a few seconds, then set it to zero again.
- Later on we can expand on the test to more steps, testing forward, backward, lateral, and yaw commands.

During these tests, record

- LowState
- LowCmd
- policy actions
- joint targets
- actual q/dq
- IMU
- faults
- timing

References

- Gallantry
 - <https://pioneeringpower.com/products/unitree-protective-gantry-for-r1-humanoid-robot>
 - <https://www.harborfreight.com/1-ton-capacity-asme-pase-certified-foldable-shop-crane-58794.html>
 - <https://www.harborfreight.com/2-ton-capacity-asme-pase-certified-foldable-shop-crane-58755.html>
- <https://www.unitree.com/R1/>
- https://github.com/unitreerobotics/unitree_sdk2_python/blob/master/example/r1/low_level/r1_low_level_example.py
- https://github.com/unitreerobotics/unitree_rl_mjlab
- https://support.unitree.com/home/en/R1_developer/about_R1
- https://github.com/unitreerobotics/unitree_sdk2